Topics to cover:

1. Dependency Injection
2. Service Layer
3. Clean Architecture

① The Problem With All logic in Routes

```
@router.Post("/")
def create_user(user: UserCreate):
    # validate
    # business logic
    # save to DB
    # send email
    # return response
```

Problem:

- Hard to test
- Hard to Reuse logic
- Routes become huge
- mixing responsibilities

② The Solution: Layers

We split code into:

- Routes → Handle HTTP (input/output)
- services → Business logic
- model → Data shape
- Repositories → DB logic

Analogy (Restaurant):

- Waiter (Route) → Take order, give Food
- chef (service) → Cooks food
- Recipe (model) → Define how dish looks
- Store room → Ingredient storage

③ What is Dependency Injection?

A dependency is something your function need to work.

Example:
- Database connection
- Logged in user
- Token validation
- Setting / Config

Without Dependency injection

```
@router.get("/user")
def get_users():
    db = DatabaseConnection()
    return db.get_all_users()
```

Problem:
- Hard to reuse
- Hard to test
- Tight coupling
- Repeated code everywhere

With Dependency Injection

```
From Fastapi import Depends
def get_db():
    return DatabaseConnection()
@router.get("/users")
def get_users(db = Depends(get_db)):
    return db.get_all_users()
```

What happens here?
- get_users() need db
- FastAPI sees Depends(get_db)
- FastAPI runs get_db()

- The result is injected into db.

Real Example : Authentication

```
def get_current_user():
    return {"username": "John"}

@router.get("/Profile")
def get_Profile(user = Depends(get_current_user)):
    return user
```

When someone calls /Profile:

1. FastAPI runs get_current_user()
2. take its return value
3. inject it into user
4. then run get_Profile()

Analogy

Think of a restaurant:

- chef need ingredients
- chef doesn't grow vegetables
- Supplier deliver them

The chef just says:
"Give me tomatoes"
that's Dependency Injection

Dependency Injection = Let Framework Provide what your Function needs.

→ Depends()
is how you ask for dependencies.

① Error Handling

From Fastapi import HTTPException

You can raise it like this:
raise HTTPException (status_code = 404,
detail = "User not Found")

FastAPI Will:
- Stop execution
- Return JSON error response
- Set correct status code

FastAPI Provide HTTPEXCEPTION to return errors

Common status codes:

Code	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
422	Validation Error
500	Internal Server error

Custom Exception Handlers
You can create your own error handler.

From Fastapi import Request
From Fastapi.responses import JSONResponse

@app.exception_handler (ValueError)
async def value_error_handler (request:
Request, exc: ValueError):

```
return JSONResponse(
    status_code = 400,
    content = {"message": str(exc)},
)
```

Now if ValueError happen:

- FastAPI catches it
- Return your custom response

Topics to cover